Práctica 2 — Reproductor de música

Estructura de Datos y Algoritmia — Curso 2026-2027
Lenguaje: C++ (estándar C++11)
Entrega: **06/11/2026 23:59** en el servidor de prácticas https://pracdlsi.dlsi.ua.es
La evaluación se hará mediante un examen práctico en ordenador (ver Sección 13).

---

1. Objetivo

Implementar **manualmente** una estructura de datos lineal con **memoria dinámica**: una **lista doblemente enlazada**.

Se trabajará con:

- **Punteros** y creación/liberación de nodos con `new`/`delete`
- La **forma canónica** con **copia profunda** (constructor de copia, operador de asignación y destructor)
- Prevención de **fugas de memoria**, **accesos inválidos** y **dobles liberaciones**
- Recorrido y modificación de una lista **doblemente enlazada** con un puntero de posición (*cursor*)
- Una aplicación completa construida sobre el TAD
- Detección de errores de memoria con **valgrind** y el **autocorrector** de la asignatura (ver el tutorial)

El resultado será un pequeño **reproductor de música** que gestiona una lista de reproducción.

> **Prohibido en esta práctica:** usar `vector`, `list`, `deque` o cualquier otro contenedor de la STL para almacenar las pistas. El objetivo es implementar la lista enlazada a mano. (Sí puedes usar `string`.)

---

2. Descripción del problema

Un reproductor mantiene una **lista de reproducción** (playlist) de pistas. Cada pista tiene un identificador único, un título y una duración en segundos. En todo momento hay una **pista actual** (la que está "sonando"), y se puede navegar a la siguiente o a la anterior de forma **circular**: después de la última se vuelve a la primera y viceversa.

---

3. Clase `Track`

3.1 Atributos (privados)

Atributo	Tipo	Restricciones
`id`	`string`	Único dentro de la playlist
`title`	`string`	Puede contener espacios
`duration`	`int`	Segundos, siempre `>= 0`

3.2 Forma canónica

```
Track();
Track(const string& id, const string& title, int duration);
Track(const Track& other);
Track& operator=(const Track& other);
~Track() = default;
```

- El constructor por defecto inicializa las cadenas vacías y `duration` a 0.
- Si el constructor recibe una duración **negativa**, se almacena 0.
- La clase no reserva memoria dinámica, por lo que el destructor puede declararse `= default`.

3.3 Getters

```
string getId() const;
string getTitle() const;
int getDuration() const;
```

3.4 Formato de duración

```
string formatDuration() const;
```

Devuelve la duración en formato `m:ss`:

- Los segundos siempre con **dos cifras** (`0:05`, `1:00`).
- Los minutos sin límite ni relleno (`65:00` para 3900 segundos).

Ejemplos: `0` → `0:00`, `5` → `0:05`, `187` → `3:07`, `3900` → `65:00`.

3.5 Operadores

```
bool operator==(const Track& other) const;
```

Dos pistas son iguales si tienen el **mismo `id`**.

```
friend ostream& operator<<(ostream& os, const Track& t);
```

Formato exacto (una sola línea, sin salto final):

```
<id> (<m:ss>) <title>
```

Ejemplo:

t01 (5:54) Bohemian Rhapsody

---

4. Clase TAD `Playlist`

Lista **doblemente enlazada** implementada a mano. La estructura `Node` se declara en la parte **privada** de la clase (solo la playlist puede acceder a los nodos); se proporciona en la plantilla:

```
struct Node {
    Track track;
    Node* prev;
    Node* next;
    Node(const Track& t) : track(t), prev(nullptr), next(nullptr) {}
};
```

El constructor de `Node` copia la pista y deja ambos enlaces a `nullptr`; es responsabilidad de la `Playlist` enlazar el nodo en su posición.

> **Los métodos de esta clase que deban recorrer la lista de nodos (como `merge`, `copy` o la función amiga `operator<<`) deberían hacerlo usando directamente los punteros, sin usar los métodos públicos `get()` y `size()` porque sería muy ineficiente.**

4.1 Atributos (privados)

Atributo	Tipo	Descripción
`head`	`Node*`	Primer nodo (`nullptr` si la lista está vacía)
`tail`	`Node*`	Último nodo (`nullptr` si la lista está vacía)
`current`	`Node*`	Pista actual ("en reproducción") (`nullptr` si la lista está vacía)
`count`	`int`	Número de pistas

La lista **no** es circular en su estructura (`head->prev` y `tail->next` son `nullptr`); la circularidad se implementa solo en `next()`/`prev()`.

4.2 Forma canónica

```
Playlist();
Playlist(const Playlist& other);
Playlist& operator=(const Playlist& other);
~Playlist();
```

- La copia debe ser **profunda**: se copian los nodos, no los punteros.
- Tras copiar, el `current` de la copia apunta al nodo de la copia que ocupa la **misma posición** que el `current` del original.

- El operador de asignación debe liberar el contenido previo y soportar la **autoasignación** (`p = p`).
- El constructor de copia y el operador asignación comparten mucho código. Deberías crear un método privado `void copy(const Playlist &other)` que puedan reutilizar esas dos funciones.
- El destructor libera **todos** los nodos.

4.3 Métodos

```
int size() const;
bool empty() const;
```

```
bool add(const Track& t);
```

Añade la pista **al final**. Si el `id` está vacío o ya existe una pista con el mismo `id`, devuelve `false` y no añade nada. Si la lista estaba **vacía**, la nueva pista pasa a ser la actual. Devuelve `true` si se añade.

```
bool insertAt(int pos, const Track& t);
```

Inserta la pista en la posición `pos`, justo antes de la pista que hubiera en esa posición. `pos` puede tomar valores desde 0 hasta `size()`, ambos incluidos; `pos == size()` equivale a añadir al final. Si `pos` está fuera de rango o el `id` está vacío o es duplicado, devuelve `false`. Si la lista estaba vacía, la nueva pista pasa a ser la actual.

```
bool removeById(const string& id);
```

Elimina la pista con ese `id`, liberando su nodo. Devuelve `false` si no existe. Si la pista eliminada era la **actual**:

- la actual pasa a ser la **siguiente**;
- si no hay siguiente, pasa a ser la **anterior**;
- si la lista queda vacía, no hay pista actual.

```
bool load(const string& filename);
```

Carga las pistas desde un fichero con el formato de la Sección 5. Si el fichero **no se puede abrir**, devuelve `false` y la lista **no se modifica**. En caso contrario, **vacía** la lista (`clear()`), añade con `add` cada línea válida (las inválidas se ignoran) y devuelve `true`. Tras la carga, la pista actual es la **primera** cargada (o no hay pista actual si no se cargó ninguna).

```
int merge(const Playlist& other);
```

Añade **al final** de esta lista todas las pistas de `other`, en su orden, usando `add` (por tanto se ignoran las que tengan un `id` ya existente). Devuelve el número de pistas realmente añadidas. `other` no se modifica.

```
void clear();
```

Elimina todas las pistas y libera toda la memoria. La lista queda vacía y usable.

```
const Track* get(int pos) const;
const Track* findById(const string& id) const;
const Track* getCurrent() const;
```

Devuelven un puntero a la pista correspondiente, o `nullptr` si no existe (posición fuera de rango, id inexistente, lista vacía).

> **Nota:** los punteros apuntan a datos internos de la lista y dejan de ser válidos cuando la pista correspondiente se elimina.

```
bool next();
bool prev();
```

Mueven la pista actual a la siguiente/anterior de forma **circular** (después de la última viene la primera y al revés). Devuelven `false` solo si la lista está vacía.

```
int totalDuration() const;
```

Suma de las duraciones de todas las pistas (en segundos).

```
friend ostream& operator<<(ostream& os, const Playlist& p);
```

Imprime una línea por pista, numeradas **desde 1**, con el formato `<n>.` seguido del de `Track::operator<<`. Cada línea empieza con un **marcador de dos caracteres**: `>` (mayor que y espacio) si es la pista actual, o dos espacios en caso contrario. Cada línea termina con salto de línea. Para la lista vacía no imprime nada.

```
  1. t01 (1:05) Uno
> 2. t02 (3:20) Dos
```

Como la clase `Node` está declarada dentro de `Playlist`, para poder usarla en este método deberías usar un código como éste:

```
ostream& operator<<(ostream& os, const Playlist& p) {
    Playlist::Node* temp = p.head;
    // resto de la función...
}
```

---

5. Formato de fichero

Es el formato que lee `Playlist::load`. Cada línea representa una pista, con 3 campos separados por `|`:

```
id|duration|title
```

Reglas de validación (mínimas)

- Si la línea no tiene exactamente **3 campos** (es decir, exactamente dos `|`): se ignora.
- Si `duration` no es un **entero válido** (según `stoi`) o es **negativo**: se ignora.

- Si alguno de los tres campos está **vacío**: se ignora.
- Si el `id` está **repetido** (ya cargado): se ignora.
- Las líneas vacías se ignoran.

Validación de números enteros en C++:

```
try {
    int duration = std::stoi(duration_str);
} catch (const std::invalid_argument& e) {
    // no es un número válido, ignorar
} catch (const std::out_of_range& e) {
    // número fuera de rango, ignorar
}
```

Ejemplo de fichero (`data/playlist_ok.txt`)

```
t01|354|Bohemian Rhapsody
t02|482|Stairway to Heaven
t03|301|Hotel California
t04|125|Blitzkrieg Bop
t05|583|Shine On
t06|205|Should I Stay
```

---

6. Aplicación `main.cc`

El programa (`player`) lee comandos desde **stdin** y escribe en **stdout**. No debe mostrar prompts ni mensajes adicionales. Las líneas vacías se ignoran sin imprimir nada.

Comandos

`LOAD <file>` Carga una playlist desde fichero mediante `Playlist::load`.

- Éxito: `OK <n>` (`n` = pistas cargadas). **Borra** el contenido anterior.
- Error al abrir: `ERROR` (y la playlist anterior **no** se modifica).
- Sin argumento: `INVALIDCMD`.

`ADD <id> <duration> <title>`

- El título es una **sola palabra** que usa **guiones bajos** (`_`) en lugar de espacios; el programa los sustituye por espacios antes de crear la pista.
- `OK` si se añade; `DUPLICATE` si ya existe ese `id`.
- Si falta algún argumento: `INVALIDCMD`.
- Si `duration` es un número negativo o no es un entero (según `stoi`): `INVALIDCMD`.

Ejemplo: `ADD t10 185 Mi_Cancion`

`DEL <id>`

- `OK` si se elimina; `NOTFOUND` si no existe.
- Sin argumento: `INVALIDCMD`.

`LIST` Imprime `SIZE <n>` seguido de la playlist (formato `operator<<`).

`PLAY` Imprime la pista actual (formato `Track::operator<<`) o `EMPTY` si la lista está vacía.

`NEXT / PREV` Mueven la pista actual (circular) e imprimen la playlist completa (formato `operator<<`, en el que el marcador `>` señala la nueva pista actual). Si la lista está vacía imprimen `EMPTY`.

`TOTAL` Imprime únicamente `<n>` (sin prefijo), donde `n` es la duración total en segundos.

`EXIT` Termina el programa sin imprimir nada.

Errores Si un comando no se reconoce o faltan argumentos: `INVALIDCMD`. Los argumentos sobrantes se ignoran.

Ejemplo de sesión Entrada:

```
LOAD data/playlist_mixed.txt
LIST
NEXT
PLAY
TOTAL
ADD t04 100 Repetida
FOO
EXIT
```

Salida:

```
OK 3
SIZE 3
> 1. t01 (5:54) Bohemian Rhapsody
  2. t04 (2:05) Blitzkrieg Bop
  3. t06 (1:03) Ultima valida
  1. t01 (5:54) Bohemian Rhapsody
> 2. t04 (2:05) Blitzkrieg Bop
  3. t06 (1:03) Ultima valida
t04 (2:05) Blitzkrieg Bop
542
DUPLICATE
INVALIDCMD
```

7. Ficheros a implementar

En `src/` se proporcionan **plantillas** con la estructura mínima de cada fichero. Debes completar:

```
src/
  Track.h
  Track.cc
  Playlist.h
  Playlist.cc
  main.cc
```

Restricciones de implementación:

- **Sin contenedores de la STL** para las pistas (nada de `vector`, `list`...). La lista enlazada se implementa con `new`/`delete`.
- No puedes usar variables globales.
- No puedes modificar las signaturas de los métodos descritos en este enunciado (puedes añadir métodos privados auxiliares).
- La lectura del fichero se hace en `Playlist::load`, no en `main.cc`; `main.cc` solo interpreta los comandos y delega en el TAD.
- El código debe compilar **sin errores ni warnings** con `g++ -std=c++11 -Wall -Wextra`.
- El programa no debe tener **fugas de memoria**: toda la memoria reservada con `new` debe liberarse exactamente una vez.

---

8. Compilación

```
make            # compila src/ -> ejecutable 'player'
make clean      # limpia binarios
```

El programa se compila con C++11 y los flags `-Wall -Wextra -g`.

Para detectar errores de memoria mientras desarrollas, ejecuta el programa con `valgrind ./player` (es lo que hace el autocorrector), o compila con *Address-Sanitizer*:

```
g++ -std=c++11 -Wall -Wextra -g -fsanitize=address src/*.cc -o
player_asan
```

---

9. Datos de prueba

Se proporcionan ficheros en `data/`:

Fichero	Contenido
`playlist_ok.txt`	6 pistas válidas
`playlist_mixed.txt`	Mezcla de líneas válidas e inválidas (3 válidas)

Fichero	Contenido
`playlist_empty.txt`	Fichero vacío (0 pistas)

---

10. Pruebas automáticas

La práctica completa se evalúa con **pruebas automáticas** organizadas en 3 grupos:

Grupo	Descripción
A	Clase `Track`
B	TAD `Playlist`
C	Aplicación CLI `player`

Las pruebas proporcionadas cubren gran parte de los casos posibles, pero es posible que haya algunos **casos límite** que no se estén probando (posiciones extremas, `current` tras borrados, copia profunda, circularidad...). Todos esos casos están descritos en este enunciado.

Debes asegurarte de que tu código cumple al 100% con las especificaciones creando tus propias pruebas del grupo Z.

Consulta el **tutorial** para aprender a ejecutar las pruebas, interpretar los resultados y depurar errores:

```
./run_tests.sh          # todas las pruebas
./run_tests.sh B        # solo el grupo B
./run_tests.sh B01      # una prueba concreta
```

> **Importante:** si `valgrind` está instalado, cada prueba se ejecuta también bajo valgrind y las fugas o accesos inválidos se marcan con el resultado **MEMORY**. Instálalo (`sudo apt install valgrind`) para obtener esa comprobación: en la corrección la gestión de memoria también se revisa.

---

11. Criterios de corrección

- Una prueba solo puntúa si **pasa** (PASS). Los resultados FAIL, ERROR, CRASH, MEMORY y TIMEOUT no puntúan.
- El código que **no compila** no puntúa en las pruebas afectadas: comprueba siempre que `make` termina sin errores ni warnings.
- Se penalizará especialmente: no respetar los formatos de salida, modificar las signaturas indicadas, el uso de variables globales, usar contenedores

STL para las pistas, las copias superficiales (compartir nodos entre listas), no liberar memoria en el destructor y no gestionar `current` al eliminar.

---

12. Entrega

Antes de entregar debes modificar el fichero `AUTHOR.txt` y añadir tu nombre completo y DNI.

Para preparar la entrega ejecuta el script `entrega.sh` en la carpeta de la práctica para generar el fichero `p2_entrega.tgz` que debes subir al servidor de prácticas:

```
./entrega.sh
# Este script creará el fichero entregable p2_entrega.tgz
```

---

13. Examen

En el examen práctico (sin Internet ni asistentes de IA) recibirás:

1. **Tu propia implementación** de esta práctica, tal y como la hayas entregado en el servidor.
2. Las **pruebas completas** de la práctica.
3. Un enunciado con una **ampliación o modificación** (típicamente una operación nueva de la lista con su comando asociado) y sus **pruebas específicas**.

Si has escrito y comprendido tu práctica, la ampliación es asequible en el tiempo disponible. Si tu práctica está incompleta, el tiempo del examen difícilmente alcanzará para completarla e implementar la ampliación.

---

14. Errores frecuentes y recomendaciones

- **Dibuja antes de programar.** Casi todos los errores de listas enlazadas se evitan dibujando los nodos y punteros antes de escribir el código.
- **Casos extremos de siempre:** lista vacía, un solo elemento, operar sobre `head`, operar sobre `tail`. Prueba los cuatro sistemáticamente.
- **`removeById` y `current`:** olvidar actualizar `current` al borrar deja un puntero colgando (*dangling*) y produce CRASH más tarde, no en el momento del borrado. Si ves un CRASH "inexplicable", busca aquí.
- **Copia profunda:** copiar `head`/`tail`/`current` tal cual comparte los nodos entre dos listas; al destruirse ambas se produce una **doble liberación**.
- **Autoasignación:** `p = p` con un `clear()` previo sin comprobación destruye los datos que ibas a copiar.
- **`insertAt(size())`** debe funcionar como `add`. No lo olvides.

- **`operator<<`:** respeta el formato exacto (marcador de dos caracteres, numeración, espacios). La mayoría de fallos del grupo C vienen de un `operator<<` incorrecto.
- **`load`:** borra el contenido previo **solo** si el fichero se abre correctamente. Cuenta únicamente las líneas válidas.
- **Paso por referencia y `const`:** los métodos que no modifican el objeto deben ser `const`; los parámetros grandes se pasan por referencia constante. El autocorrector no compilará si faltan los `const`.
- Trabaja de forma **incremental**: `Track` (grupo A) → `add`, `get`, `size`, destructor → resto del grupo B (incluidos `load` y `merge`) → `main.cc` (grupo C).